MERGING TWO SORTED ARRAYS

Using Pointers and the Two-Pointer Technique

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads the sizes and elements of two sorted, variable-length integer arrays, arr1[] and arr2[], whose elements are in ascending order. Both arrays are passed to a function named merge_sorted_arrays() using their addresses (pointers). The function must:

- Merge arr1[] and arr2[] into a third array, merged[].
- Maintain the ascending order of the elements in merged[].
- Preserve duplicate elements, if any.
- Display the elements of merged[] separated by spaces.
- Use the two-pointer technique to merge the arrays efficiently.

2. Approach and Concepts Used

2.1 Two-Pointer Technique

Since both input arrays are already sorted, there is no need to sort the combined data again. One pointer (p1) walks through arr1[] and another pointer (p2) walks through arr2[]. At every step the two current elements are compared and the smaller one is copied to merged[]; only the pointer of the array that supplied the element is moved forward. When one array is exhausted, the remaining elements of the other array are copied directly, because they are already in ascending order.

2.2 Handling Duplicates

When *p1 and *p2 are equal, the condition (*p1 <= *p2) copies the element of arr1[] first, and the equal element of arr2[] is copied in the next step. Hence every occurrence is retained in merged[] and no duplicate is lost.

2.3 Pointers and Dynamic Memory

Because the array sizes are not fixed, memory for arr1[], arr2[] and merged[] is allocated at run time with malloc() and released with free(). Array elements are accessed with pointer notation (*p, p++, arr + i) and the arrays are passed to the function as int * parameters.

3. Algorithm

1. Start.
2. Read the size n1 and then n1 elements of arr1[].
3. Read the size n2 and then n2 elements of arr2[].
4. Call merge_sorted_arrays(arr1, n1, arr2, n2), passing the base addresses of both arrays.
5. Inside the function, allocate merged[] of size n1 + n2 and set p1 = arr1, p2 = arr2, pm = merged.
6. While both p1 and p2 have elements left: if *p1 <= *p2, copy *p1 to merged[] and advance p1; otherwise copy *p2 and advance p2. Advance pm each time.
7. Copy any remaining elements of arr1[], then of arr2[], to merged[].
8. Print all elements of merged[] separated by spaces.
9. Free the allocated memory and Stop.

Complexity

Time complexity: O(n1 + n2), as each element is visited exactly once.

Space complexity: O(n1 + n2) for the merged[] array.

4. Source Code

```
/*
 * Program : Merge Two Sorted Arrays
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Description:
 *   Reads two sorted integer arrays (ascending order) of variable length
 *   and merges them into a third sorted array using the two-pointer
 *   technique. Duplicates are preserved. Arrays are passed to the
 *   function using pointers (their addresses).
 */

#include <stdio.h>
#include <stdlib.h>

/* Function prototype */
void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2);

int main(void)
{
    int n1, n2, i;
    int *arr1, *arr2;

    /* Read size and elements of the first array */
    printf("Enter size of Array 1: ");
    if (scanf("%d", &n1) != 1 || n1 < 0)
    {
        printf("Invalid size for Array 1.\n");
        return 1;
    }

    arr1 = (int *)malloc((n1 > 0 ? n1 : 1) * sizeof(int));
    if (arr1 == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements of Array 1 (ascending order): ", n1);
    for (i = 0; i < n1; i++)
        scanf("%d", arr1 + i);

    /* Read size and elements of the second array */
    printf("Enter size of Array 2: ");
    if (scanf("%d", &n2) != 1 || n2 < 0)
    {
        printf("Invalid size for Array 2.\n");
        free(arr1);
        return 1;
    }

    arr2 = (int *)malloc((n2 > 0 ? n2 : 1) * sizeof(int));
    if (arr2 == NULL)
    {
        printf("Memory allocation failed.\n");
        free(arr1);
        return 1;
    }

    printf("Enter %d elements of Array 2 (ascending order): ", n2);
    for (i = 0; i < n2; i++)
        scanf("%d", arr2 + i);

    /* Pass the addresses of both arrays to the function */
    printf("Output:\n");
    merge_sorted_arrays(arr1, n1, arr2, n2);

    free(arr1);
    free(arr2);
    return 0;
}

/*
```

```
 * merge_sorted_arrays()
 * Merges two sorted arrays into merged[] using the two-pointer technique
 * and displays the merged array.
 */
void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2)
{
    int *merged;
    int *p1 = arr1;            /* pointer traversing arr1   */
    int *p2 = arr2;            /* pointer traversing arr2   */
    int *end1 = arr1 + n1;     /* one past the last of arr1 */
    int *end2 = arr2 + n2;     /* one past the last of arr2 */
    int *pm;                   /* pointer traversing merged */
    int total = n1 + n2;

    merged = (int *)malloc((total > 0 ? total : 1) * sizeof(int));
    if (merged == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }
    pm = merged;

    /* Compare the current elements and copy the smaller one */
    while (p1 < end1 && p2 < end2)
    {
        if (*p1 <= *p2)
            *pm++ = *p1++;
        else
            *pm++ = *p2++;
    }

    /* Copy remaining elements of arr1, if any */
    while (p1 < end1)
        *pm++ = *p1++;

    /* Copy remaining elements of arr2, if any */
    while (p2 < end2)
        *pm++ = *p2++;

    /* Display merged[] separated by spaces */
    for (pm = merged; pm < merged + total; pm++)
        printf("%d ", *pm);
    printf("\n");

    free(merged);
}
```

5. Sample Output

The program was compiled using gcc with the -Wall -Wextra flags (no warnings) and tested on the three given examples. User input is shown in bold blue.

Test Case 1

```
Enter size of Array 1: 4
Enter 4 elements of Array 1 (ascending order): 1 3 5 7
Enter size of Array 2: 4
Enter 4 elements of Array 2 (ascending order): 2 4 6 8
Output:
1 2 3 4 5 6 7 8
```

Test Case 2 (with duplicates)

```
Enter size of Array 1: 4
Enter 4 elements of Array 1 (ascending order): 1 2 2 4
Enter size of Array 2: 3
Enter 3 elements of Array 2 (ascending order): 2 3 5
Output:
1 2 2 2 3 4 5
```

Test Case 3 (arrays of different lengths)

```
Enter size of Array 1: 3
Enter 3 elements of Array 1 (ascending order): 1 4 7
Enter size of Array 2: 5
Enter 5 elements of Array 2 (ascending order): 2 3 5 6 8
Output:
1 2 3 4 5 6 7 8
```

6. Conclusion

The program successfully merges two sorted arrays of variable length into a single sorted array using the two-pointer technique. Pointers are used to pass the arrays to merge_sorted_arrays() and to traverse them, duplicates are preserved, and the merge is done in linear time without re-sorting. The results for all three given examples match the expected output.